

Calcolatori Elettronici

8 luglio 2024

Teoria

Esercizio 1 (12 punti): Progettare una rete sequenziale in grado di interpretare sequenze infinite di cifre codificate utilizzando il *codice di Gray* a 3 bit. Ciascuna cifra viene inviata in input alla rete sequenziale un bit alla volta, partendo dal bit meno significativo. Dopo aver letto una cifra, la rete sequenziale fornisce in output il valore "sì" se la cifra letta è pari, in tutti gli altri casi fornisce in output il valore "no". Realizzare il circuito equivalente dell'equazione minimizzata di un flip flop di stato.

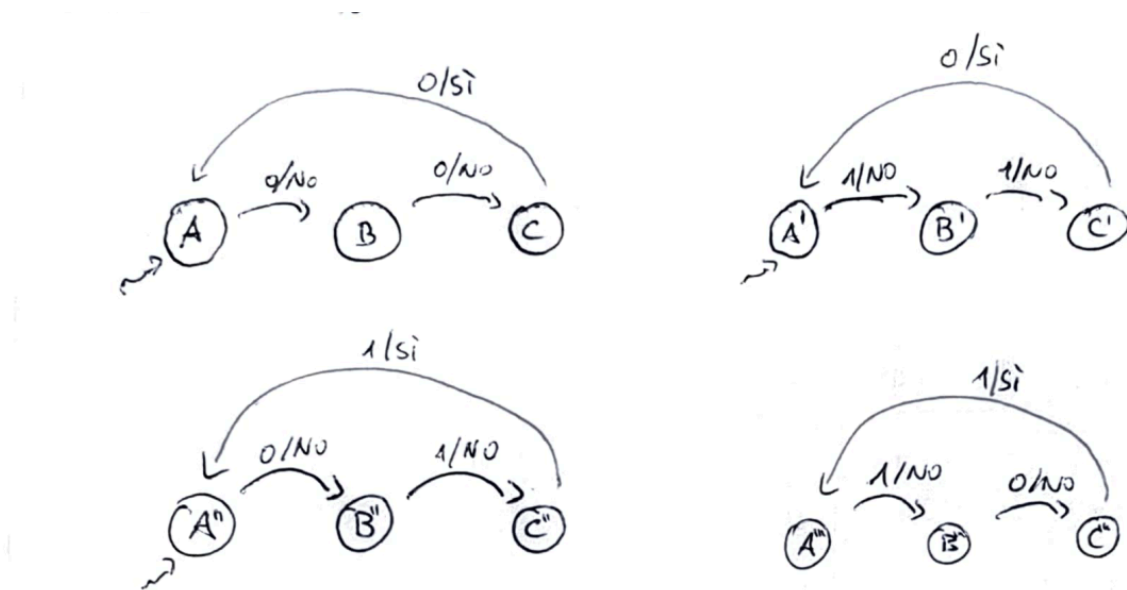
Esercizio 2 (10 punti): Disegnare l'architettura interna del processore z64. Descrivere il microcodice atto a supportare l'esecuzione dell'istruzione `addb $1, %a1`, compresa la fase di fetch.

Esercizio 3 (8 punti): Discutere il funzionamento interno di una memoria DRAM. Per quale motivo è necessaria l'operazione di refresh? Quali sono le sue conseguenze sulle prestazioni dell'esecuzione dei programmi?

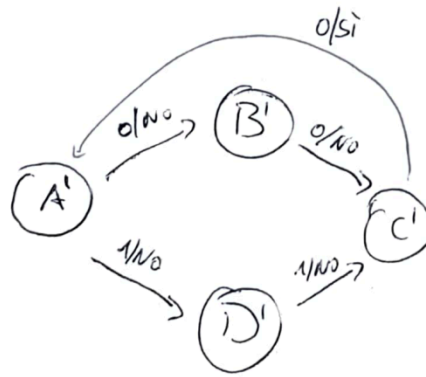
Soluzioni

Esercizio 1

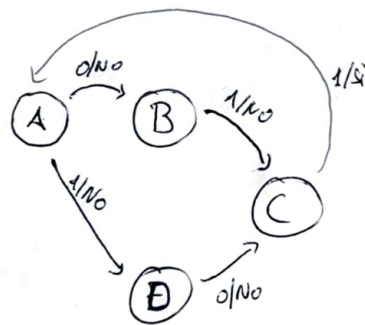
La rete sequenziale deve riconoscere i numeri pari di 3 bit codificati con il codice di Gray, pertanto le sequenze che devono fornire in output "sì" sono 000, 011, 110, 101, ricordandosi che queste vengono lette dal bit meno significativo al bit più significativo. Per approssimare l'esercizio, possiamo innanzitutto abbozzare 4 automi differenti che accettino, singolarmente, le quattro sequenze di interesse:



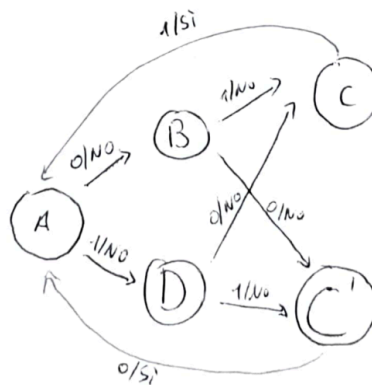
Osservando i primi due automi, si può notare che lo stato A e lo stato A' sono equivalenti, così come lo stato C e C'. Questi possono quindi essere fusi insieme in due coppie di stati:



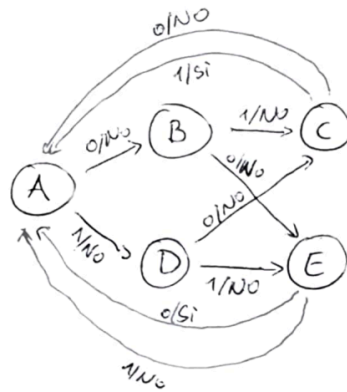
Si può fare lo stesso ragionamento sui secondi automi, fondendo insieme, rispettivamente, gli stati A'' e A''' e C'' e C''' :



A questo punto, osservando questi due automi, si può notare l'equivalenza tra gli stati A e A' , B e B' , C e C' . Unendo insieme gli automi si ottiene:



Per completare l'automa, è necessario identificare ora tutti quegli stati che non hanno un numero di uscite pari al numeri di input ammissibili (ossia 2) e completare le transizioni. Si ottiene quindi il seguente automa:



Utilizzando come codifica degli stati la seguente: $A = 000$, $B = 001$, $C = 010$, $D = 011$, $E = 100$, si ottiene la seguente tabella degli stati e delle transizioni:

y_2	y_1	y_0	x	y_2'	y_1'	y_0'	z
0	0	0	0	0	0	1	0
0	0	0	1	0	1	1	0
0	0	1	0	1	0	0	0
0	0	1	1	0	1	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	0	1	0	0
0	1	1	1	1	0	0	0
1	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0

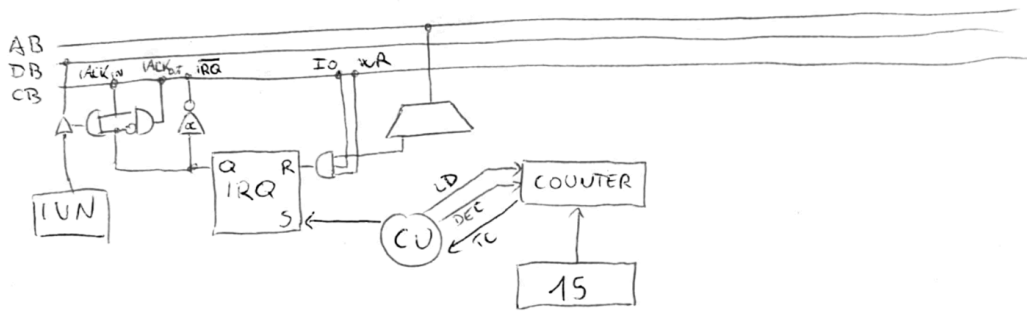
in cui le configurazioni non indicate sono *don't care conditions*. Volendo realizzare il circuito equivalente dell'equazione di eccitazione di y_1 , si può procedere alla minimizzazione con mappa di Karnaugh:

y_1'

$y_2 y_1$	00	01	11	10
00	0	0	-	0
01	1	0	-	0
11	1	0	-	-
10	0	1	-	-

che porta all'equazione $y_1' = \overline{y_2}y_1x + y_1y_0\overline{x}$, il cui circuito equivalente è il seguente:

La periferica **TIMER** è una periferica asincrona. Essa invia una richiesta di interruzione ogni 15 secondi, indipendentemente da quello che avviene nel resto del sistema. La sua interfaccia può essere la seguente:



Una possibile implementazione del software è la seguente.

```

1  .org 0x800
2  .data
3      .equ ANTENNA_STATUS, 0x1
4      .equ ANTENNA_LAT, 0x2
5      .equ ANTENNA_LON, 0x3
6      .equ TIMER_IRQ, 0x4
7      .equ PULSANTE_IRQ, 0x5
8      .equ MEMORIA, 0x6
9      vettore: .fill 1024, 8, 0
10     indice: .long 0
11     registrazione: .byte 0 # Flag per modalità registrazione (0 = pausa, 1 = registrazione)
12     timer_interrupt_flag: .byte 0
13
14 .text
15 main:
16     sti
17     .main_loop:
18         cmpb $1, timer_interrupt_flag # Verifica il flag di interruzione del timer
19         jnz .main_loop
20         movb $0, timer_interrupt_flag # Reset del flag di interruzione del timer
21         cmpb $1, registrazione # Controlla se siamo in modalità registrazione
22         jnz .main_loop
23         cmpl $1024, indice # Verifica se il vettore è pieno
24         call program_dma
25         call read_antenna
26         jmp .main_loop
27
28 .driver 0 # Driver del pulsante
29     xorb $1, registrazione
30     outb %al, $PULSANTE_IRQ
31     iret
32
33 .driver 1 # Driver del timer
34     movb $1, timer_interrupt_flag
35     outb %al, $TIMER_IRQ
36     iret
37
38 read_antenna:
39     outb %al, $ANTENNA_STATUS
40     .bw:
41     inb $ANTENNA_STATUS, %al
42     btb $0, %al
43     jnc .bw
44     movzbl indice, %rcx
45     inl $ANTENNA_LAT, %eax
46     movl %eax, vettore(,%rcx,8)
47     inl $ANTENNA_LON, %eax

```

```

48     movl %eax, vettore+4(,%rcx,8)
49     addq $1, %rcx
50     movl %ecx, indice
51     ret
52
53 program_dma:
54     movq $vettore, %rsi
55     movq $1024, %rcx
56     movw $MEMORIA, %dx
57     cld
58     outsb
59     movl $0, indice
60     ret

```

Prova di laboratorio

Si vuole realizzare un programma in C che acquisisce frasi da terminale, ne analizza le parole e le organizza in 16 liste singolarmente concatenate, in cui ciascun nodo segue la seguente definizione:

```

1 struct nodo {
2     char *parola;
3     unsigned contatore;
4 };

```

Dopo l'inserimento di una frase, il programma la suddivide in parole, utilizzando come carattere di divisione tra le parole gli spazi (anche ripetuti) ed ignorando eventuali segni di interpunzione. Ciascuna parola è gestita nel modo seguente. Il primo carattere di ogni parola è utilizzato per determinare in quale lista concatenata la parola deve essere memorizzata, sfruttando i 4 bit meno significativi del primo carattere per selezionare una delle 16 liste disponibili. Identificata la lista corretta, questa viene attraversata: se un nodo con la stessa parola viene trovato, il contatore delle occorrenze viene incrementato; altrimenti, viene creato un nuovo nodo con il contatore impostato a 1.

Il programma continua ad acquisire frasi fino a quando non viene inserita la stringa `termina`. A quel punto, vengono stampati i valori dei contatori per ciascuna parola e il programma termina.

Soluzione

La seguente è una possibile soluzione all'esercizio.

```

1 #include <stdbool.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <ctype.h>
6
7 #define NUM_LISTS 16
8 #define HASH_FUNCTION(parola) ((parola)[0] & 0xF)
9 #define MAX_WORD_LEN 256
10 #define MAX_LINE 1024
11
12 struct nodo {
13     char *parola;
14     unsigned contatore;
15     struct nodo *next;
16 };
17
18 struct nodo *liste[NUM_LISTS] = {NULL};
19

```

```

20 static void inserisci_parola(char *parola) {
21     unsigned int pos = HASH_FUNCTION(parola);
22     struct nodo **current = &liste[pos];
23
24     // Cerca la parola nella lista
25     while (*current != NULL) {
26         if (strcmp((*current)→parola, parola) == 0) {
27             (*current)→contatore++;
28             return;
29         }
30         current = &((*current)→next);
31     }
32
33     // Se la parola non è stata trovata, crea un nuovo nodo
34     struct nodo *new_node = (struct nodo *)malloc(sizeof(struct nodo));
35     if (new_node == NULL) {
36         perror("malloc");
37         exit(EXIT_FAILURE);
38     }
39     new_node→parola = strdup(parola);
40     if (new_node→parola == NULL) {
41         perror("strdup");
42         free(new_node);
43         exit(EXIT_FAILURE);
44     }
45     new_node→contatore = 1;
46     new_node→next = NULL;
47
48     *current = new_node; // Inserisci il nuovo nodo alla fine della lista
49 }
50
51 // NOTA: l'utilizzo di strtok() ridurrebbe sensibilmente la lunghezza di questa funzione.
52 // Tuttavia, non essendo una funzione che presento a lezione, la soluzione non ne fa uso.
53 void processa_frase(char *frase) {
54     char parola[MAX_WORD_LEN];
55     int j = 0;
56
57     for (int i = 0; frase[i] != '\0'; i++) {
58         if (isspace(frase[i]) || ispunct(frase[i])) {
59             if (j > 0) {
60                 parola[j] = '\0';
61                 for (int k = 0; k < j; k++) {
62                     parola[k] = tolower(parola[k]);
63                 }
64                 inserisci_parola(parola);
65                 j = 0;
66             }
67         } else {
68             parola[j++] = frase[i];
69         }
70     }
71
72     // Gestione della parola finale se non seguita da spazio o punteggiatura
73     if (j > 0) {
74         parola[j] = '\0';
75         for (int k = 0; k < j; k++) {
76             parola[k] = tolower(parola[k]);
77         }
78         inserisci_parola(parola);
79     }
80 }
81
82 void stampa_liste() {
83     for (int i = 0; i < NUM_LISTS; i++) {
84         struct nodo *current = liste[i];
85         while (current != NULL) {

```

```

86         printf("Parola: %, Contatore: %u\n", current->parola, current->contatore);
87         current = current->next;
88     }
89 }
90 }
91
92 void libera_memoria() {
93     for (int i = 0; i < NUM_LISTS; i++) {
94         struct nodo *current = liste[i];
95         while (current != NULL) {
96             struct nodo *temp = current;
97             current = current->next;
98             free(temp->parola);
99             free(temp);
100         }
101     }
102 }
103
104 int main() {
105     char buffer[MAX_LINE];
106
107     while (true) {
108         printf("Inserisci una frase (o 'termina' per terminare): ");
109         if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
110             break;
111         }
112         buffer[strcspn(buffer, "\n")] = 0;
113
114         if (strcmp(buffer, "termina") == 0) {
115             break;
116         }
117
118         processa_frase(buffer);
119     }
120
121     stampa_liste();
122     libera_memoria();
123
124     return 0;
125 }
126

```